Département EEA – Faculté Sciences et Ingénierie



M1 EEA – Parcours Systèmes et Microsystèmes Embarqués

Rapport de Bureau d'Études VHDL / SOPC

Fonction gestion du verin

Année universitaire 2026–2027

Rédigé par **ARIEF Audy** et **NSHUTI NKURIKIYINKA Fidèle**

Binôme : **B6**

27 octobre 2026

Table des matières

1 Introduction et cahier des charges **2**

2 Architecture fonctionnelle **2**

3 Conception VHDL **3**
3.1 Génération de la PWM 3
3.2 Base de temps 3
3.3 Interface avec le MCP3201 3
3.4 Contrôle des butées et sens de rotation 4

4 Simulation sous ModelSim **4**

1 Introduction et cahier des charges

Ce travail porte sur la conception, la simulation et la validation en VHDL du circuit `gestion_verin`, qui pilote le vérin de la barre franche. Le circuit est destiné à la carte **DE0-Nano** et doit à terme être relié au processeur **Nios II** par le biais du bus **Avalon**.

Le cahier des charges impose quatre fonctions principales :

— La gestion de la modulation de largeur d'impulsion (**PWM**) du moteur ;
— Le contrôle des butées de fin de course du vérin ;
— La gestion du convertisseur analogique-numérique (**CAN MCP3201**), qui effectue la recopie de position de la barre ;
— L'interface avec le bus Avalon du processeur Nios II.

Note d'avancement : L'interface Avalon n'a pas encore été réalisée au cours de cette session. Les signaux `freq`, `duty`, `butee_g` et `butee_d` sont pour l'instant définis comme des valeurs fixes au sein du code source.

2 Architecture fonctionnelle

Le circuit se décompose en deux grands groupes de fonctions distinctes : l'acquisition de la position de la barre d'une part, et la commande du moteur d'autre part.

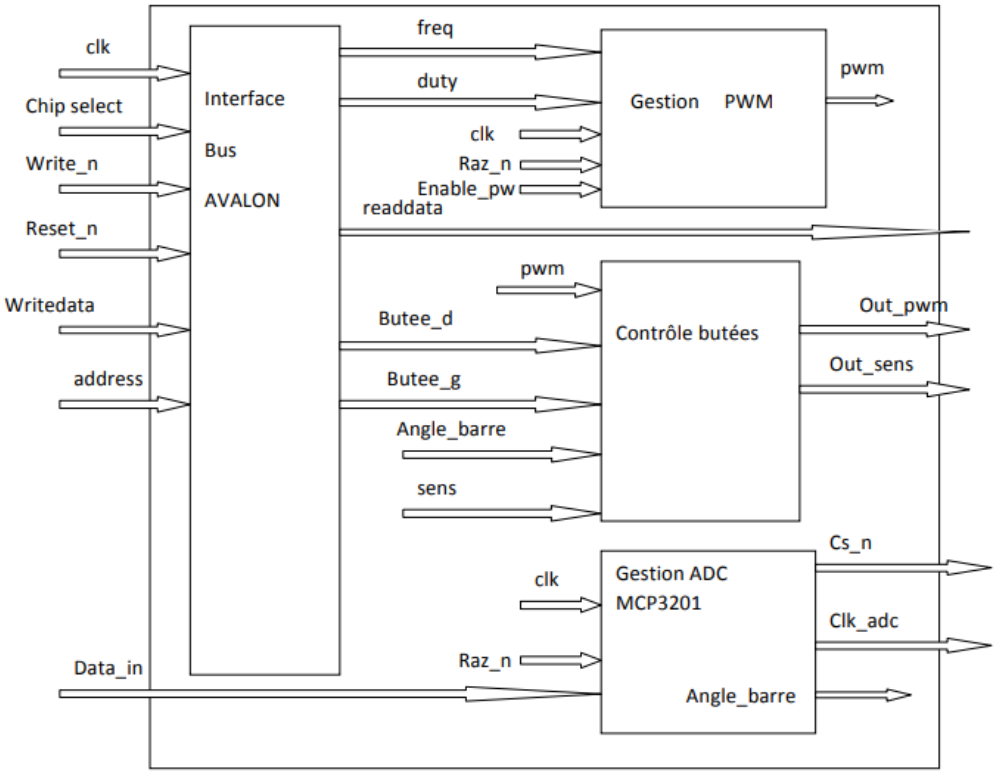


FIGURE 1 – schema foctionnel gestion verrin.

Fonction (*process*)	**Rôle**	**Entrées / Sorties principales**
`divide`	Fixe la fréquence de la PWM	`freq` → `counter`
`compare`	Fixe le rapport cyclique et génère le signal PWM	`counter`, `duty` → `pwm_signal`
`controle_butees`	Compare la position aux butées et lève les drapeaux	`angle_mem`, `butee_g`, `butee_d` → `fin_course_g`, `fin_course_d`
`gene_start_conv`	Déclenche une conversion toutes les 100 ms	`start_conv`
`gene_1M`	Génère l'horloge à 1 MHz pour le CAN	`signal_1M`
`pilote_adc`	Machine à états finis (*idle* / conversion)	`start_conv`, `fin_conversion` → `ep`
`compt_fronts` / `rec_dec`	Comptent les fronts de `clk_adc` et décalent les 12 bits	`dout` → `angle_mem`

TABLE 1 – Description des fonctions et processus du circuit.

Le tableau 1 détaille le rôle opérationnel ainsi que les entrées/sorties de chaque processus mis en œuvre. La sortie de commande globale `pwm` n'est coupée que si une butée matérielle est atteinte **et** que le signal de direction `sens` pousse activement le vérin vers cette même butée. Une fois en butée, le système maintient la sécurité tout en permettant au moteur de repartir immédiatement dans le sens opposé.

3 Conception VHDL

3.1 Génération de la PWM

Le compteur interne `counter` s'incrémente de manière synchrone à chaque coup d'horloge globale jusqu'à atteindre la valeur limite `freq`, avant de repasser à `0x0000`. Le signal intermédiaire `pwm_signal` est forcé à '1' lorsque le compteur passe par zéro, et retombe à '0' dès qu'il franchit le seuil `duty`.

Avec les constantes de configuration provisoires définies par :

— `freq` = `0xC350` (soit 50 000 coups d'horloge)

— `duty` = `0x6000` (soit 24 576 coups d'horloge)

On obtient une fréquence de hachage de la PWM de 1 kHz associée à un rapport cyclique d'environ 49 %.

3.2 Base de temps

Sous une horloge système de 50 MHz, le bloc `gene_1M` implémente un compteur modulo 50 (comptage de 0 à 49) pour émettre une impulsion calibrée toutes les microsecondes (μs).

En parallèle, le processus `gene_start_conv` compte de 0 à 4 999 999 pour générer un top de déclenchement `start_conv` toutes les 100 ms exactement. Cette période temporelle étant choisie comme un multiple strict de 50 cycles, chaque début de conversion s'aligne de façon déterministe sur la même phase de `signal_1M`. Cela permet d'assurer un premier front d'horloge parfaitement propre et exempt de glitchs à destination du circuit intégré MCP3201.

3.3 Interface avec le MCP3201

Le composant MCP3201 est un convertisseur analogique-numérique (CAN) 12 bits à communication série synchrone (type SPI). La machine à états finis `pilote_adc` bascule en phase de scrutation active dès l'apparition de l'impulsion `start_conv`. Ce changement d'état provoque la mise à l'état bas de la ligne de sélection `cs` (Chip Select) et autorise la diffusion de l'horloge d'échantillonnage `clk_adc` (1 MHz).

Le bloc logique quantifie ensuite le protocole d'échange :

1. Le bloc `compt_fronts` dénombre les fronts actifs de 0 à 14.
2. Le registre à décalage `rec_dec` capture sériellement chaque échantillon binaire présent sur la broche `dout`.
3. En fin de cycle de conversion, la trame de 12 bits reconstruite est verrouillée dans le registre interne `angle_mem`, puis recopiée vers le port de sortie externe `angle_barre_sortie`.

3.4 Contrôle des butées et sens de rotation

Le bloc `controle_butees` opère une surveillance continue en comparant la variable `angle_mem` aux valeurs limites `butee_g` et `butee_d`. Les indicateurs d'état `fin_course_g` et `fin_course_d` sont mutuellement exclusifs par construction. L'algorithme effectue l'évaluation à partir du signal interne `angle_mem`.

La coupure de protection s'articule selon l'équation logique suivante : la sortie `pwm` est forcée à '0' uniquement si l'on se trouve dans l'une des deux configurations critiques suivantes :

— Butée gauche atteinte alors que le signal de direction exige un mouvement à gauche (`sens` = '0') ;
— Butée droite atteinte alors que le signal de direction exige un mouvement à droite (`sens` = '1').

4 Simulation sous ModelSim

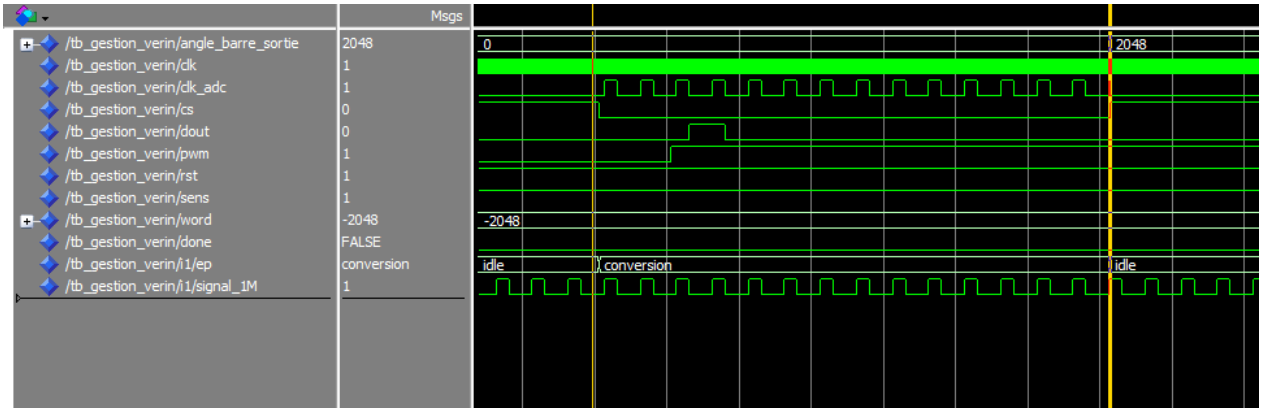


FIGURE 2 – Simulation de gestion du verrin.